AOSP 进阶丛书 · 第十二册 · ANDROID 13 (API 33)

媒体管线
音频与相机

AudioFlinger 混音车间与 AudioPolicy 调度室、三档延迟与焦点约定、Camera2 请求-结果流水线与 ZSL/多摄、MediaCodec 双队列与 Codec2、MediaDrm 与 AAudio。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：媒体是三条"实时流水线" 2

第一章 音频管线：从 write() 到扬声器 2

第二章 相机管线：请求-结果流水线 4

第三章 编解码与 DRM：双队列模型 6

第四章 设计决策九问 7

附录 A 观测实操 7

附录 B 快问快答（14 条） 8

附录 C 自检题 8

第五章 音频框架的前世与生态位 8

附录 D 动手实验室（半天量） 9

第六章 AudioFlinger::createTrack 逐段走读：策略在前，车间在后 10

第七章 AF 线程家族与 FastMixer：一个混音周期的内心戏 14

第八章 一次 AudioTrack.write 的出声之旅 18

第九章 CameraService 接入仲裁与 Camera3Device 建立 22

教科书习题集（分级） 28

术语表 30

排障手册：媒体管线问题定位 32

媒体管线核心速查表 35

本册速览

音频：AF 混音车间+AP 调度室 · 三档延迟(normal/fast/MMAP)
焦点是约定：策略裁决不拦混音 · Offload 省电与线程家族
相机：Camera2 请求-结果流水线 · ZSL/多摄/逻辑相机
编解码：MediaCodec 双队列+Codec2 · MediaDrm L1/L3 与 AAudio
附录 D 五实验室：延迟实测/路由现场/焦点攻防/in-flight/Offload

前言：媒体是三条"实时流水线"

媒体子系统的共性是**实时性**：音频按周期混音（20ms 级）、相机按帧出图（33ms 级）、编解码按 buffer 流转——它们都不是"请求-响应"而是"流水线+节奏"。本册讲三条管线：**音频**（AudioFlinger 混音车间 + AudioPolicy 调度室）、**相机**（Camera2 请求-结果流水线）、**编解码**（MediaCodec/Codec2 双队列），末章 MediaDrm 与书11 信任链衔接。

源码基线不变（本树 Android 13）。主角目录：`frameworks/av/services/audioflinger`（AF）、`frameworks/av/services/audiopolicy`（AP）、`frameworks/av/services/camera/libcameraservice`（CameraService）、`hardware/interfaces/{audio,camera}`（HAL）。

第一章 音频管线：从 write() 到扬声器

设计洞察 · 学习目标

1. 能画出音频四段（AudioTrack→AF 混音→AP 策略→HAL/DAC）并标注延迟三档位；2. 能说出 AF 与 AP 的分工边界（执行与决策）；3. 能区分 normal/fast track/MMAP 三档延迟的适用场景与代价；4. 能解释焦点为什么是约定而非强制



图 1 音频四段：AudioTrack→AudioFlinger 混音→AudioPolicy 策略→HAL/DAC，含延迟三档位与焦点语义

1.1 逐段注释（真实锚点）

cpp frameworks/av（骨架+讲解注释）

```
// App 侧：AudioTrack::set（media/libaudioclient/AudioTrack.cpp:500/531/849）
// 讲解注释①：set 声明格式（采样率/声道/usage），随后 write()
// 写入环形 buffer；FAST track 则共享内存直投（低延迟档）。
// 服务端：AudioFlinger::createTrack（audioflinger/AudioFlinger.cpp:1023）
// 讲解注释②：AF（:306 构造，audioserver 进程）先问 AP：
// getOutputForAttr ——这个流（usage=media/通话/铃）该去哪个设备？
// 讲解注释③：MixerThread::threadLoop_write（Threads.cpp:4908）：
// 每周期把该输出上所有 Track 混音写 HAL；
// DirectOutputThread（:2827）不混直通（高音质流）。
// 策略侧：AudioPolicyManager（audiopolicy/manager/）
// 讲解注释④：路由（插耳机切换）/音量（按 stream type）/焦点（互抢裁决）
// ——AP 只决策，AF 执行；两者进程内协作、接口分离。
```

1.3 AudioFlinger 的线程家族（读码地图）

text Threads.cpp 的线程谱系（audioserver 内）

```
PlaybackThread // 普通回放母类：周期=frameCount/sampleRate
├─ MixerThread(:4908) // 混音：多 Track→一路 PCM→HAL
│  └─ OffloadThread // 压缩流（MP3/AAC）不解码直送 DSP——省电主力
└─ DirectOutputThread(:2827) // 不混直通（高音质/低延迟单流）
RecordThread // 录音：反向 PCM 从 HAL 取
MmapThread // 8+：MMAP 共享内存直投（AAudio 低延迟的底座）
// 讲解注释：每"输出设备+打开配置"组合一条线程——
// 蓝牙+扬声器同时出声=两条 MixerThread 并行；dumpsys af 逐线程可见。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释媒体管线的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：Offload 为什么省电

普通路径：MP3→CPU 解码→PCM→混音→HAL（CPU 全程醒）。Offload：压缩流直接交 DSP，CPU 可睡——长音乐播放功耗差 30%+。代价：无法混音（独占输出）+seek 延迟——所以"通知声叠在音乐上"时策略层会切回普通路径，这个切换本身也是 AP 的决策。

1.4 音量与路由：策略层的两张表

text AudioPolicyManager 的决策素材（读码归纳）

```
音量表：STREAM_(MUSIC/RING/ALARM/VOICE_CALL/…) × 设备(扬声器/耳机/BT)
```

```
→ setStreamVolume 只改表；真实音量在 AudioFlinger 施加
（软件音量在混音时乘增益；硬件音量直写 HAL amplifier）
路由表：设备状态机（插拔/连接 BT 触发重路由）
→ 每流 usage→设备 的映射规则（引擎可配：engineconfigurable/）
→ "导航说话时音乐闪避（duck）"= 策略引擎的注意力仲裁
// 讲解注释：dumpsys media.audio_policy 的前两段就是这两张表的实时投影。
```

设计洞察 · 逐段精读：三个延迟档与选型

normal（binder 写+混音周期，100ms 级）→ fast track（共享内存+低周期 ~20ms）→ **MMAP no-IRQ**（8+：硬件时钟直驱 <10ms，AAudio 首选）。音游/通话回声消除选 MMAP；普通提示音 normal 足矣——延迟是用功耗换的，不是免费午餐。

1.2 焦点：约定而非强制

App 的 requestAudioFocus 是**约定**：策略侧裁决并广播 LOSS/GAIN，但混音不物理停流——"被夺焦后停音乐"是 App 守约的行为。系统只在铃声/通话场景强制降权（duckt）。面试高频："两个 App 同时放歌会怎样"——都在放（物理层不拦），体验层靠焦点协商（所以流氓 App 能"抢音"）。

第二章 相机管线：请求-结果流水线

设计洞察 · 学习目标

1. 能画出 Camera2 请求-结果流水线并说明异步请求-结果对这一核心抽象；2. 能说出 openCamera→connectHelper→Client 创建的服务端三段链；3. 能解释 ZSL/多摄/HDR+ 为什么只有流水线架构才能做；4. 会用 dumpsys media.camera 做预览黑屏的三查



图 2 Camera2 四段：openCamera(:825)→CameraService connectHelper(:1750)→HAL 3A/ISP→onCaptureCompleted，Camera1/2/X 对照与权限隐私

2.1 逐段注释（真实锚点）

java CameraManager/CameraService（骨架+讲解注释）

```
// App：CameraManager.openCamera（core/.../CameraManager.java:825/862/927）
// 讲解注释①：异步 open——回调拿到 CameraDevice；
// createCaptureSession 绑定输出 Surface（预览/拍照/分析各自流）。
// 服务端：CameraService（libcameraservice/CameraService.cpp:145 构造）
// 讲解注释②：connectHelper（:1750）：客户端接入仲裁——
// 高优客户端（SystemUI 人脸）可踢低优；权限与"前台可见"检查。
// 讲解注释③：请求流水线：setRepeatingRequest（预览=无限重复）+
// capture（拍照=插一个高分辨率请求）；按 HAL in-flight 节奏逐个下发。
// HAL：hardware/interfaces/camera/provider/（2.4-2.6，AIDL 化进行中）
// 讲解注释④：3A（AE/AF/AWB）与 ISP 在厂商 HAL；每帧回
// TotalCaptureResult（图像+metadata"怎么拍的"）——
// 异步请求-结果对是 Camera2 的核心抽象。
```

2.3 多摄与 ZSL（Camera2 能力的兑现）

text 为什么只有流水线架构能做（读码归纳）

```
ZSL 零快门：环形缓冲持续捕帧，按快门时回溯取帧——
需要逐帧控制（新旧架构里 Camera1 做不到，只能"提前开预览"）
多摄同步：逻辑相机（ID 背后多个物理头）同一请求下发
——深度/虚化=两路帧时间戳对齐（HAL 保证 sensor sync）
HDR+：短曝多帧+算法合成——逐帧改曝光参数的流水线是前提
// 讲解注释：CameraManager.getCameraIdList 里带 LOGICAL_MULTI_CAMERA
// 能力集的 ID 即逻辑相机；物理头经 getPhysicalCameraIds 暴露。
```

设计洞察 · 逐段精读：Camera1/2/X 一句话

Camera1 黑箱（takePicture 一锤子）；Camera2 把 ISP 开成**可编程请求队列**（逐帧改参数/多 Surface/ZSL/RAW）；CameraX 是 Camera2 的易用层（生命周期感知），不是新管线。

2.2 排障三查

`dumpsys media.camera`（客户端/会话/in-flight 全表）；预览黑屏查三处：会话 Surface 配置、流组合是否被 HAL 支持（camera2 的 supported stream combinations）、后台限制（无前台服务开相机即拒）。指示器（绿点）由 SystemUI 监听 camera use 事件强制点亮——隐私不可关闭。

第三章 编解码与 DRM：双队列模型

设计洞察 · 学习目标

1. 能画出 MediaCodec 双队列（input/output buffer 池）的同步/异步两种用法；2. 能说出解码直投 Surface 省掉哪一跳；3. 能解释 Codec2 相对 OpenMAX 的演进点与迁移现状；4. 能区分 DRM L1/L3 安全级并说清与书11 KeyMint 的关系



图 3 MediaCodec 双队列（input/output buffer 池）、Codec2 组件链、MediaDrm 安全通路与 AAudio

text MediaCodec 形状（读码归纳）

```
App ↔ [inputBuffer 队列] ↔ 编解码器 ↔ [outputBuffer 队列] ↔ App
· configure(format, surface) → start() → dequeue/queue 循环（或异步回调）
· 解码直投 Surface（buffer 不过 App 手）——视频播放省一跳
· 底层：OpenMAX → Codec2（组件链，参数运行时可调）迁移中
MediaDrm（书11 呼应）：
```

- DRM 会话密钥走 KeyMint 同源硬件路径；Secure Decoder 全链加密
- L1（硬件全链，4K）/L3（软件，受限）安全级

AAudio（8+ 音频新世代）：MMAP no-IRQ 硬件时钟直驱，stream builder+状态机；底层仍由 AF/AP 管理新旧并存。三板斧观测：`dumpsys media.{codec,audio_flinger,camera}`。

第四章 设计决策九问

设计洞察 · 学习目标

1. 能对每问给出"问题→备选方案→决策→代价"四步式答案；2. 能把九问归入三类动机（崩溃隔离/演进成本/安全）；3. 能用本书源码锚点为至少三问举证；4. 能反向出题——从一条设计决策推出它牺牲了什么

- **为什么音频混音在独立 audioserver 进程？** 崩溃隔离（App 坏歌不断）+ 实时调度统一管理（混音线程 SCHED_FIFO 集中一处）。
- **为什么策略（AP）与执行（AF）分离？** 设备拓扑/音效策略高频定制（厂商各异），混音引擎相对稳定——切分线画在"变化频率"上。
- **为什么焦点是约定？** 强制停流会破坏合法场景（导航+音乐混音）；把最终裁量权给最知情的 App。
- **为什么 Camera2 改成请求流水线？** 逐帧可控才能做 ZSL/HDR+/多摄；黑箱模型封死了这些能力。
- **为什么相机结果带 metadata？** 后处理（人脸/夜景算法）需要"这帧怎么拍的"；只给图等于丢参数。
- **为什么 HAL 版本众多（2.4-2.6+AIDL 并行）？** 相机能力爆炸式演进+厂商碎片化——版本化是无奈也是必需（书1 HAL 三代呼应）。
- **为什么 MediaCodec 用双队列池化？** buffer 分配昂贵（图形内存）；池化+复用把分配从热路径挪走。
- **为什么 DRM 解码要 Secure Decoder？** 高清内容授权要求明文不可见（L1）；与 fscrypt 同一哲学——密文到最后一刻。
- **为什么 AAudio 不替换 AudioTrack？** 生态兼容成本；新旧并存让新 API 只服务低延迟场景（增量演进优于推倒重来）。

附录 A 观测实操

目标	命令	看点
音频全表	dumpsys media.audio_flinger	Track/线程/延迟模式
音频策略	dumpsys media.audio_policy	路由/设备/焦点栈
相机全表	dumpsys media.camera	客户端/会话/in-flight

目标	命令	看点
编解码	dumpsys media.codec	实例/格式/buffer
权限	appops get <pkg> CAMERA	使用记录（绿点数据源）

附录 B 快问快答（14 条）

- AF 与 AP 的分工一句话？——混音车间与调度室。
- createTrack 在哪？——AudioFlinger.cpp:1023。
- 三个延迟档？——normal/fast track/MMAP no-IRQ。
- 焦点是强制的吗？——约定，混音层不拦。
- 插耳机音乐怎么切的？——AP 设备状态机重路由。
- Camera2 核心抽象？——异步请求-结果对。
- connectHelper 干什么？——客户端接入仲裁（:1750）。
- 预览本质？——setRepeatingRequest 无限重复。
- CameraX 是新管线吗？——不是，Camera2 易用层。
- 相机绿点能关吗？——不能（SystemUI 强制）。
- MediaCodec 双队列？——input/output buffer 池化复用。
- Codec2 是什么？——组件链式新一代编解码框架。
- DRM L1/L3 区别？——硬件全链 vs 软件受限。
- AAudio 底层？——仍由 AF/AP 管理（并存非替代）。

附录 C 自检题

- 默画音频四段与 20ms 周期内发生的事。
- 焦点 LOSS 后系统做了什么、没做什么?
- 一次拍照从 capture() 到 onCaptureCompleted 的全链（含 in-flight 语义）。
- MediaCodec 直投 Surface 为什么省一跳?
- DRM L1 与书11 KeyMint 的关系。

第五章 音频框架的前世与生态位

设计洞察 · 学习目标

1. 能按年代列出 AudioTrack/AAudio/媒体3 三代门面及其底座关系；2. 能解释"换门面不换车间"的分层价值；3. 能说清蓝牙 A2DP 与 USB Audio 在 AP 里的抽象位置；4. 能用"蓝牙没声音"案例演示策略层→协议栈的两段定位法

5.1 从 AudioSystem 到 AAudio 的十年

text 音频 API 世代（读码归纳）

```
2011-：AudioTrack/AudioRecord（libaudioclient）
——推拉模型，set/write；至今仍是大多数 App 的底座
2014-：AAudio（8.0，O）
——stream builder+状态机；MMAP no-IRQ 低延迟路径
2017-：SoundPool/MediaPlayer 门面化（内部仍 AudioTrack）
——ExoPlayer/媒体3 统一播放器层（App 侧库，非系统）
系统侧不变：AF 混音 + AP 策略 + HAL
// 讲解注释：API 换代三层但管线一层——"换门面不换车间"，
// 这正是分层的价值：应用层进化不迫害底层重写。
```

5.2 声卡之外的"音频设备"：蓝牙与 USB

路由策略的特殊分支：**蓝牙 A2DP**（音频走协议栈不经声卡，delay reporting 支持音画同步）与 **USB Audio Class**（外接 DAC 直挂 ALSA）。两者都由 AP 统一抽象为"输出设备"——App 无感。排障："连了蓝牙没声音"先查 AP 设备状态机（`dumpsys media.audio_policy` 的 connected devices），再查 A2DP 栈（书10 呼应：蓝牙也是独立 Mainline 模块）。

附录 D 动手实验室（半天量）

实验一"音频延迟实测"：AudioTrack 写正弦+示波器 App（或 tap-to-tone）测 normal vs PERFORMANCE_MODE_LOW_LATENCY 差值；AAudio MMAP 对比（需 8+ 设备）。

实验二"路由现场"：播放中插拔耳机，`dumpsys media.audio_policy` 前后对比（路由表/活动设备变化）；开勿扰后铃声/媒体音量的策略差异。

实验三"焦点攻防"：两个 App 同时 requestAudioFocus(GAIN)，观察广播次序；一个不守约继续播——验证"混音不停"（第一章 1.2 的实证）。

实验四"in-flight 之旅"：`dumpsys media.camera` 连续采样预览中的 inFlightCount；拍照瞬间 spike（高分辨率请求入队）——第二章流水线实证。

实验五"Offload 省电"：播放本地 MP3 闲置 10 分钟记录耗电，对比"关 offload"（开发者选项）——第一章 1.3 的数字版。

第六章 AudioFlinger::createTrack 逐段走读：策略在前，车间在后

设计洞察 · 学习目标

1. 能复述 createTrack 的五段式结构（身份校验→会话分配→策略路由→车间建轨→句柄返回）；2. 能说出 AudioPolicyManager 内部 getOutputForAttr 的四层漏斗裁决顺序；3. 能解释 TrackHandle/Track/Client 三者的引用关系与跨进程边界；4. 能对照 dumpsys 输出辨认 ioHandle、portId、sessionId 三类句柄

6.1 链路位置：一次 AudioTrack.set() 的服务端第一跳

第一章 1.1 用 ::fig aud-pipeline 画了音频四段，第五章讲了 API 门面的十年变迁；本章把图的中间一段放大到函数级。前置链路是：App 进程里 `AudioTrack::set()`（AudioTrack.cpp:500/531/849）收集格式与属性，随后 `createTrack_l()`（media/libaudioclient/AudioTrack.cpp:1883）把参数打包成 `media::CreateTrackRequest` 经 binder 投递到 audioserver——落地处就是本章入口 `AudioFlinger::createTrack`（AudioFlinger.cpp:1023）。后置链路是第七章的线程家族与第八章的 write 运转之旅。

为什么这个函数值得逐行读？因为它是"**策略与执行分离**"的物理交汇点：AF 自己不决定这条流去哪个输出设备，它做的第一件事就是把球踢给 AudioPolicyManager——AudioFlinger.cpp:1079 的 `AudioSystem::getOutputForAttr()`。策略裁决错了，后面混音写得再对也是"在错误的耳机里出声"。

6.2 走读 createTrack：五段式

cpp frameworks/av/services/audioflinger/AudioFlinger.cpp:1023-1221（骨架+讲解注释）

```
status_t AudioFlinger::createTrack(const media::CreateTrackRequest& _input,
media::CreateTrackResponse& _output)
{
CreateTrackInput input = VALUE_OR_RETURN_STATUS(CreateTrackInput::fromAidl(_input));
CreateTrackOutput output; // 讲解注释①：AIDL 请求/响应结构——T 上 AF 主接口
// 已 Aidl 化，旧 parcel 版只剩兼容路径
// ---- 第一段：身份校验（不可信调用者的参数一律改写）----
const uid_t callingUid = IPCThreadState::self()->getCallingUid();
if (!isAudioServerOrMediaServerUid(callingUid)) {
ALOGW_IF(clientUid != callingUid,
"%s uid %d tried to pass itself off as %d",
__FUNCTION__, callingUid, clientUid); // 讲解注释②：普通 App 冒充他人 uid
adjAttributionSource.uid = ...; // 不会崩、不会拒，只会被强制改回——
clientUid = callingUid; // "纠偏而非惩罚"是 AF 的安全风格
}
// ---- 第二段：会话号分配（效果链的"房间号"）----
if (sessionId == AUDIO_SESSION_ALLOCATE) {
sessionId = (audio_session_t) newAudioUniqueId(AUDIO_UNIQUE_ID_USE_SESSION);
} // 讲解注释③：同 sessionId 的 Track 与 Effect
// 挂同一条 EffectChain——音乐 App 的均衡器
```

```
// 就是靠这个号"追上"它的 Track 的
// ---- 第三段：问策略（本函数最重要的一跳）----
lStatus = AudioSystem::getOutputForAttr(&localAttr, &output.outputId, sessionId,
&streamType, adjAttributionSource, &input.config,
input.flags, &output.selectedDeviceId, &portId,
&secondaryOutputs, &isSpatialized);
// 讲解注释④：AF 不决策去哪个输出——AudioSystem
// 的 binder 转手给 AudioPolicyService（6.3）
if (lStatus != NO_ERROR || output.outputId == AUDIO_IO_HANDLE_NONE) {
ALOGE("createTrack() getOutputForAttr() return error %d ...", lStatus);
goto Exit; // 讲解注释⑤：无路可走即建轨失败——不存在
// "路由不到就随便找条线程放着"的兜底
}
// ---- 第四段：参数校验与车间建轨 ----
if (!audio_is_output_channel(input.config.channel_mask)) { ... goto Exit; }
{
Mutex::Autolock _l(mLock); // 讲解注释⑥：AF 全局锁——所有建轨串行化，
// 这也是建轨延迟抖动的一个来源
PlaybackThread *thread = checkPlaybackThread_l(output.outputId); // :1112
if (thread == NULL) { ... goto Exit; } // ioHandle→线程 映射不存在即坏值
client = registerPid(clientPid); // 按 pid 记账：内存配额与异常退出后的回收依据
track = thread->createTrack_l(client, streamType, localAttr, &output.sampleRate,
input.config.format, input.config.channel_mask,
&output.frameCount, ..., sessionId, &output.flags, ...,
portId, ...); // 讲解注释⑦：真正的建轨与 FAST
// 门槛裁决发生在"车间"线程侧——
// Threads.cpp:2229，第七章 7.3 展开
if (lStatus == NO_ERROR) {
updateSecondaryOutputsForTrack_l(track.get(), thread, secondaryOutputs);
} // 讲解注释⑧：副输出（多设备齐播）失败不回滚
// 主路——注释明言"desync 直到 track 销毁"
if (lStatus == NO_ERROR && effectThread != NULL) {
if (moveEffectChain_l(sessionId, effectThread, thread) == NO_ERROR) { ... }
} // 讲解注释⑨：EffectChain 跟 session 走、不跟
// 线程走——效果链跨输出"搬家"在此完成
}
// ---- 第五段：门面返回 ----
output.audioTrack = new TrackHandle(track); // 讲解注释⑩：:1214——客户端拿到的是壳（6.4）
_output = VALUE_OR_FATAL(output.toAidl());
Exit:
if (lStatus != NO_ERROR && output.outputId != AUDIO_IO_HANDLE_NONE) {
AudioSystem::releaseOutput(portId); // 讲解注释⑪：失败路径必须归还策略侧登记——
} // 否则 AP 的客户端表泄漏、路由状态漂移
return lStatus;
}
```

设计洞察 · 逐段精读：身份段的两条守则

第一守则：调用者自称什么不算数，binder 内核记录的 callingUid/callingPid 才算数（AudioFlinger.cpp:1043/1059）；第二守则：只有 audioserver/mediaserver 自己（isAudioServerOrMediaServerUid）可以代第三方申报身份——这就是系统服务替 App "代理创建"的合法通道。对照第三章 DRM 与第二章相机的同名套路：media 三件套的信任模型完全一致。

6.3 策略侧：getOutputForAttr 的四层漏斗

策略裁决的主体在 AudioPolicyService（进程内直接持有 AudioPolicyManager 对象，无二次 binder）：AudioPolicyInterfaceImpl.cpp:320 收到请求后原样转手 mAudioPolicyManager->getOutputForAttr。APM 内部是一个四层漏斗。

cpp frameworks/av/services/audiopolicy/managerdefault/AudioPolicyManager.cpp:1282-1346（骨架+讲解注释）

```
status_t AudioPolicyManager::getOutputForAttr(const audio_attributes_t *attr, ...,
audio_port_handle_t *portId, ..., bool *isSpatialized)
{
if (*portId != AUDIO_PORT_HANDLE_NONE) {
return INVALID_OPERATION; // 讲解注释①：portId 是 AP 发放的，客户端不许自备
}
const audio_port_handle_t requestedPortId = *selectedDeviceId;
const sp<DeviceDescriptor> requestedDevice =
mAvailableOutputDevices.getDeviceFromId(requestedPortId); // 讲解注释②：App 可用
// setPreferredDevice 指名设备，
// 但只认"当前在线"的设备
// 四层漏斗：层1 指名设备 → 层2 PolicyMix（远端路由）→ 层3 产品策略引擎 → 层4 按 profile 开输出
status_t status = getOutputForAttrInt(&resultAttr, output, session, attr, stream, uid,
config, flags, selectedDeviceId, &isRequestedDeviceForExclusiveUse,
secondaryOutputs != nullptr ? &secondaryMixes : nullptr, outputType, isSpatialized);
// 讲解注释③：主体在 :1124——PolicyMix 优先级最高，
// 其次是 mEngine 的 usage→strategy→device 映射
...
*portId = PolicyAudioPort::getNextUniqueId(); // 讲解注释④：发放 portId（客户端级句柄）
sp<SwAudioOutputDescriptor> outputDesc = mOutputs.valueFor(*output);
sp<TrackClientDescriptor> clientDesc =
new TrackClientDescriptor(*portId, uid, session, resultAttr, clientConfig,
sanitizedRequestedPortId, *stream,
mEngine->getProductStrategyForAttributes(resultAttr),
toVolumeSource(resultAttr), ...);
// 讲解注释⑤：一次接入=一条 Client 描述符：
// 音量源（哪个 STREAM 曲线）与产品策略在这
// 里定死，之后音量调节查的就是这张表
outputDesc->addClient(clientDesc); // 讲解注释⑥：:1346——AP 侧登记完成；dumpsys
// media.audio_policy 的客户端列表即来源于此
return NO_ERROR;
}
```

漏斗主体的前半段值得单独看一眼——getOutputForAttrInt（AudioPolicyManager.cpp:1124）里层2/层3 的交接：

cpp frameworks/av/services/audiopolicy/managerdefault/AudioPolicyManager.cpp:1124-1196（骨架+讲解注释）

```
status_t AudioPolicyManager::getOutputForAttrInt(..., audio_output_flags_t *flags, ...)
{
DeviceVector outputDevices;
const sp<DeviceDescriptor> requestedDevice =
mAvailableOutputDevices.getDeviceFromId(requestedPortId);
...
status_t status = getAudioAttributes(resultAttr, attr, *stream);
if (status != NO_ERROR) { return status; } // 讲解注释①：先统一身份证——旧 stream
// type 规范化成 attributes（usage/content）
```

```
*stream = mEngine->getStreamTypeForAttributes(*resultAttr); // 反查旧流型只为兼容回填
// ---- 层2：动态路由（PolicyMix，媒体投影/车载分区的钩子层）----
sp<AudioPolicyMix> primaryMix;
status = mPolicyMixes.getOutputForAttr(*resultAttr, uid, *flags, primaryMix,
secondaryMixes);
// 讲解注释②：普通 App 此处为空——命中的
// 是 RemoteDisplay 类远端播放场景
bool usePrimaryOutputFromPolicyMixes = requestedDevice == nullptr && primaryMix != nullptr;
// 讲解注释③：指名设备优先级高于 PolicyMix
// ——两条主路先排序再走
if ((usePrimaryOutputFromPolicyMixes || ...)
&& !audio_is_linear_pcm(config->format)) {
return BAD_VALUE; // 讲解注释④：动态路由只支持 PCM——压缩流
// 禁入投影路径（日志原话）
}
if (usePrimaryOutputFromPolicyMixes) {
...
status = openDirectOutput(*stream, session, config,
(audio_output_flags_t)(*flags | AUDIO_OUTPUT_FLAG_DIRECT), ...);
// 讲解注释⑤：PolicyMix 命中仍可能要新开
// direct 输出——策略与 profile 协作
if (policyDesc != nullptr) {
*output = policyDesc->mIoHandle;
return NO_ERROR; // 命中动态路由：不再问引擎
}
}
// Virtual sources must always be dynamically or explicitly routed
if (resultAttr->usage == AUDIO_USAGE_VIRTUAL_SOURCE) {
return BAD_VALUE; // 讲解注释⑥：虚拟源没有引擎兜底——
// 路由不到就是错，不猜
}
// ---- 层3：产品策略引擎 ----
outputDevices = mEngine->getOutputDevicesForAttributes(*resultAttr, requestedDevice,
false);
// 讲解注释⑦：usage→设备 的真正裁决——
// enginedefault 是默认实现，厂商可换
// configurable 引擎（engineconfigurable/）
...
}
```

漏斗的第四层藏在 `getOutputForAttrInt` 内部：常规 PCM 流直接命中共享混音输出；但当请求是 DIRECT/OFFLOAD、或采样率声道数超出共享输出能力时，会走到 `openDirectOutput`（AudioPolicyManager.cpp:1354）。它的第一道门槛是"不值得开"判定：非 DIRECT 标志 + 线性 PCM + 采样率不超上限 + 双声道以内 → 直接 `NAME_NOT_FOUND` 回共享 MixerThread——系统绝不给一个普通通知音开独占输出。第二道门槛是 offload 反悔机制：`mEffects.isNonOffloadableEffectEnabled()` 为真时拒绝创建 offload 流，因为 DSP 解码路径挂不上这条全局音效。

> **设计洞察 · 设计洞察：策略裁决的全部痕迹都在句柄里**
> getOutputForAttr 返回四个产物：outputId（ioHandle，线程级）、portId（客户端级）、streamType（旧流型，兼容层）、secondaryOutputs（副输出）。之后 AF 的所有行为都由这些句柄驱动，AP 不再参与逐 buffer 决策——策略"开火后不管"，执行层全速运转。这正是 1.4 两张表（音量表/路由表）能在运行时换内容而不惊动混音引擎的原因。

6.4 TrackHandle：跨进程的门面

cpp frameworks/av/services/audioflinger/Tracks.cpp:333-341（骨架+讲解注释）

```
AudioFlinger::TrackHandle::TrackHandle(const sp<AudioFlinger::PlaybackThread::Track>& track)
: BnAudioTrack(),
mTrack(track)
{
} // 讲解注释①：壳构造——持 sp 强引用保活 Track，
// 客户端进程多久，Track 就活多久
AudioFlinger::TrackHandle::~TrackHandle() {
mTrack->destroy(); // 讲解注释②：壳亡则轨灭——AF 从不把 Track 裸指针
} // 交给 binder 层，生命周期闭环在门面内
```

TrackHandle 的每个 binder 方法都是一行转发（start→`mTrack->start()`、stop→`mTrack->stop()`……），它存在的唯一意义是把"线程内对象"包装成"可跨进程对象"。客户端侧拿到的 `IAudioTrack` 代理（AudioTrack.cpp:2025 `mAudioTrack = output.audioTrack`）与 AF 内 Track 一一对应；track 重路由或输出重配置导致控制块失效时（CBLK_INVALID），客户端用 `restoreTrack_l`（AudioTrack.cpp:2891）重新走一遍 createTrack——注意不是复用旧句柄，而是"换一根新轨"。

> **设计洞察 · 与书3 的交汇：这里是一次 binder 十跳的完整样本**
> CreateTrackRequest 进、CreateTrackResponse 出，一个来回；之后 start/stop 各一个来回；而真正每 20ms 的音频数据零次 binder——走的是 cblk 共享内存（第八章 8.2）。"控制面走 binder、数据面走共享内存"是 AOSP 多媒体子系统的统一模式，相机侧的 CameraMetadata 与编解码侧的 buffer 提交莫不如此。

第七章 AF 线程家族与 FastMixer：一个混音周期的内心戏

> **设计洞察 · 学习目标**
> 1. 能画出 PlaybackThread 子类谱系并说出每条线程的触发条件与归属流；2. 能复述 PlaybackThread::threadLoop 主循环六步（等待→prepare→mix→effects→write→remove）；3. 能解释 FastMixer 静态/动态模式与 fast track 的七条准入门槛；4. 会从 dumpsys media.audio_flinger 读出线程类型、underrun 计数与 FastMixer 状态

7.1 链路位置：线程是"车间"，Track 是"工单"

第六章的 createTrack 结束于 `thread->createTrack_l(...)`——工单送进了某个车间；本章看车间本体。1.3 的读码地图给出了线程谱系，这里补上 C++ 层的真实继承关系（Threads.h）：`class PlaybackThread : public ThreadBase`（:832）、`class MixerThread : public PlaybackThread`（:1393）、`class DirectOutputThread : public PlaybackThread`（:1492）、`class OffloadThread : public DirectOutputThread`（:1567）、`class DuplicatingThread : public MixerThread`（:1627）、`class RecordThread : public ThreadBase`（:1704）、`class MmapThread : public ThreadBase`（:1977）。注意两个容易被面试题问倒的细节：**OffloadThread 继承的是 DirectOutputThread 而非 MixerThread**（压缩流天然不混音）；**DuplicatingThread 继承 MixerThread**（先混好，再复制给多个下游 OutputTrack）。

线程创建时机在 AudioPolicyManager 侧：AP 为一条新输出调用 mpClientInterface->openOutput → AF 打开 HAL 输出流并 new 一条对应类型的线程。所以"每输出设备+打开配置一条线程"不是 AF 主动决定的，而是策略层 profile 匹配的结果——这是第六章 6.3 漏斗第四层的下游。

7.2 主循环走读：threadLoop 的六步

cpp frameworks/av/services/audioflinger/Threads.cpp:3680-3935（骨架+讲解注释）

```
bool AudioFlinger::PlaybackThread::threadLoop()
{
Vector< sp<Track> > tracksToRemove;
cacheParameters_l(); // 讲解注释①：缓存 mFrameCount/mSampleRate 等，
// 热路径禁止反复加锁读取
while (!exitPending()) {
// ---- 第 1 步：无活可干则睡（standby 驱动）----
if (mActiveTracks.isEmpty() && mConfigEvents.isEmpty()) {
// we're about to wait, flush the binder command buffer
IPCThreadState::self()->flushCommands(); // 讲解注释②：睡前把 binder 命令缓冲
... // 冲掉——避免"已响应却还欠着命令"
if (waitNoShutdown_l()) { continue; } // 等 mWaitWorkCV 条件变量被 start 唤醒
}
{ // mLock scope
// ---- 第 2 步：盘点工单——所有 Track 的综合状态 ----
mMixerStatus = prepareTracks_l(&tracksToRemove); // :3885
mActiveTracks.updatePowerState(this);
// ---- 第 3 步：锁死效果链清单再混音 ----
lockEffectChains_l(effectChains); // 讲解注释③：防"混着混着音效链被删"——
// 效果创建/删除必须等这一周期结束
activeTracks.insert(activeTracks.end(), mActiveTracks.begin(), mActiveTracks.end());
} // mLock scope ends // 讲解注释④：数据面出锁——混音本身不持
// 大锁，这是实时性与一致性的折中点
if (mBytesRemaining == 0) {
mCurrentWriteLength = 0;
if (mMixerStatus == MIXER_TRACKS_READY) {
// ---- 第 4 步：混音（子类各自实现）----
threadLoop_mix(); // :3931——MixerThread::threadLoop_mix :5034
} else if (...非 DRAIN...) {
```

```
threadLoop_sleepTime(); // 没数据就逐级拉长睡眠时间（退避省电）
}
}
// ---- 第 5 步：写 HAL（第八章放大）----
ret = threadLoop_write(); // :4111 → PlaybackThread::threadLoop_write :3297
...
// ---- 第 6 步：清尸与效果处理 ----
threadLoop_removeTracks(tracksToRemove); // :3254——stop/pause/underrun 剔除的
... // Track 在锁外统一收尾
}
}
```

`prepareTracks_l` 是整个混音周期的"裁判"：它汇总每条 active track 的 framesReady、音量渐变、underrun 状态，返回 `MIXER_TRACKS_READY`（至少一条有足够数据）或 `MIXER_TRACKS_DISABLED/NO_TRACKS`（全部没料）。裁判结果直接决定第 4 步是混音、写零垫底还是逐步拉长睡眠进入 standby——`dumpsys media.audio_flinger` 里每个线程的 `underruns` 计数正是这里累计的。

> **设计洞察 · 逐段精读：为什么不持锁混音**
>
> 第 3 步之后 mLock 就放开了（讲解注释④），混音与 HAL write 在锁外跑。原因：混一个 20ms 周期可达数百微秒，若持 AF 全局锁，任何 binder 线程的 setVolume/setParameters 都会阻塞混音线程——实时线程被非实时请求卡住就是咔哒声的来源。代价是 AF 锁的粒度设计极其谨慎（第六章 createTrack 的 mLock 段因此格外短）。

7.3 MixerThread 与它的 FastMixer

MixerThread 构造函数决定"要不要装 FastMixer"：

cpp frameworks/av/services/audioflinger/Threads.cpp:4678-4730（骨架+讲解注释）

```
AudioFlinger::MixerThread::MixerThread(const sp<AudioFlinger>& audioFlinger,
AudioStreamOut* output, audio_io_handle_t id, bool systemReady, type_t type, ...)
: PlaybackThread(audioFlinger, output, id, type, systemReady, mixerConfig),
mFastMixerFutex(0), mMasterMono(false) { ... }
if (type == DUPLICATING) {
return; // 讲解注释①：DuplicatingThread 混完写下游
// OutputTrack，不需要 FastMixer/HAL sink
}
mOutputSink = new AudioStreamOutSink(output->stream); // NBAIO 封装 HAL 输出流
...
bool initFastMixer;
switch (kUseFastMixer) {
case FastMixer_Never: initFastMixer = false; break;
case FastMixer_Always: initFastMixer = true; break;
case FastMixer_Static: // 讲解注释②：Threads.cpp:203 默认值
case FastMixer_Dynamic:
initFastMixer = mFrameCount < mNormalFrameCount; // 讲解注释③：HAL 周期 < 混音
// 周期才值得装——低延迟输出
//  （如深管 deep buffer 之外
// 的 fast profile）才开
}
```

```
if (initFastMixer) {
...
// create a MonoPipe to connect our submix to FastMixer
... // 讲解注释④：MonoPipe=无锁单写单读 FIFO，
// 普通混音线程写、FastMixer 线程读
}
```

kUseFastMixer = FastMixer_Static（Threads.cpp:203）是编译期默认：FastMixer 在构造时决定装不装，运行期不再拆除。装了之后，普通混音线程的职责变成"把慢速混好的数据写入 MonoPipe"，FastMixer 以 SCHED_FIFO 实时优先级按 HAL 小周期从 MonoPipe 取数再精混，两线程经无锁 FIFO 解耦。MixerThread::threadLoop_write（Threads.cpp:4908）的头 20 行就是"唤醒 FastMixer"的仪式：把 FastMixerState::MIX_WRITE 命令经 StateQueue push 过去，冷态时用 futex 唤醒（syscall(__NR_futex, ..., FUTEX_WAKE_PRIVATE, 1)）。FastMixer 干活的主体在 FastMixer.cpp:352 onWork()：遍历 fast track 位掩码（__builtin_ctz 找最低位），逐条取音量、查 framesReady、underrun 时 mMixer->disable(name) 垫零。

那 App 怎么进 FastMixer？答案是必须先拿到 **fast track** 资格。服务器端终裁在 PlaybackThread::createTrack_l（Threads.cpp:2282）：

cpp frameworks/av/services/audioflinger/Threads.cpp:2282-2341（骨架+讲解注释）

```
// client expresses a preference for FAST, but we get the final say
if (*flags & AUDIO_OUTPUT_FLAG_FAST) {
if (
audio_is_linear_pcm(format) && // 讲解注释①：必须线性 PCM——压缩流
// 无混音可言，fast 无意义
(channelMask == (mChannelMask | mHapticChannelMask) || ...) && // ②：声道配置
// 要么完全一致要么无需昂贵的下混
(sampleRate == mSampleRate) && // 讲解注释③：必须等于硬件采样率——
// 重采样进不了 fast 路径
hasFastMixer() && // 讲解注释④：这条线程装了 FastMixer
(mFastTrackAvailMask != 0) // 讲解注释⑤：还有空余 slot——位掩码
// 管理名额，fast track 数量有限
) {
if (sharedBuffer == 0) {
...
frameCount = max(frameCount, mFrameCount * sFastTrackMultiplier); // ⑥：流式
// track 至少要按"整周期×倍数"给 buffer
}
{ // scope for mLock
for (audio_session_t session : { AUDIO_SESSION_DEVICE, ... }) {
sp<EffectChain> chain = getEffectChain_l(session);
if (chain.get() != nullptr) {
chain->checkOutputFlagCompatibility(flags);
}
} // 讲解注释⑦：全局/输出级音效在挂时——
} // 降级为普通路径（音效 DSP 进 fast
// 路径会破坏周期预算）
} else {
ALOGD("AUDIO_OUTPUT_FLAG_FAST denied: ...");
*flags = (audio_output_flags_t)(*flags & ~AUDIO_OUTPUT_FLAG_FAST); // 任何一条不满足
} // ——静默降级，App 只会收到回调通知
}
```

> **设计洞察 · 设计洞察：fast track 是"资格制"不是"申请制"**
> 七条门槛（线性 PCM/声道一致/采样率一致/装了 FastMixer/slot 空闲/帧数达标/无全局音效）里没有一条看 App 的身份——但 slot 位掩码在混音高峰先到先得。客户端侧还有前置自查（AudioTrack.cpp:1896 起：共享 buffer 或 callback/obtain 模式才放行 FAST flag）。所以"用了 PERFORMANCE_MODE_LOW_LATENCY 也可能回落普通路径"是常态而非 bug——dumpsys 里 track 的 flags 才是终裁结果。

线程家族分工速查（都在 Threads.h，行号为类定义锚点）：

线程	继承自	典型流	混音性	锚点
MixerThread	PlaybackThread	音乐/系统提示音	多轨混音+可选 FastMixer	Threads.h:1393
OffloadThread	DirectOutputThread	MP3/AAC 长播放	不混，压缩直送 DSP	Threads.h:1567
DirectOutputThread	PlaybackThread	HDMI 多声道/低延迟独占	不混直通	Threads.h:1492
DuplicatingThread	MixerThread	双设备齐播	混一次复制多份	Threads.h:1627
RecordThread	ThreadBase	录音	反向取流	Threads.h:1704
MmapThread	ThreadBase	AAudio MMAP	周期由硬件时钟驱	Threads.h:1977

7.4 从 dumpsys 印证

`dumpsys media.audio_flinger` 每个线程块头部形如 `Output thread 0xf... type 0 (MIXER)`，后面跟 `Format/Frame count/...`、每条 track 的 `Usage/Content/Flags/Underruns`。第七章所有机制都能在输出里对号入座：type 字段即线程谱系（0=MIXER/1=direct/2=offload…）、underruns 即 prepareTracks_l 的累计成绩单、fast track 的 mFastIndex 反映 slot 分配。做第八章实验前先看懂这张表，排障手册的场景二直接用它定位。

第八章 一次 AudioTrack.write 的出声之旅

> **设计洞察 · 学习目标**
> 1. 能完整叙述 write() 之后数据经过的每一段内存（用户数组→cblk 环形区→getNextBuffer→sinkBuffer→HAL）；2. 能指出全链 binder 与 memcpy 各发生几次、发生在谁与谁之间；3. 能解释 underrun 的成因、三种症状与 DirectOutputThread 的垫零处理；4. 能用 write/obtainBuffer 的返回值区分"缓冲满""会阻塞""服务死亡"

8.1 链路位置：把四段图压成一段旅程

第六章建好了轨（策略裁决），第七章认好了车间（线程家族），本章让声音真正流起来。全程对照 ::fig aud-pipeline：App 的 `write()`（第 1 段尾巴）→ 共享内存 cblk → AF 线程（第 2 段）→ HAL（第 3 段）→ DAC（第 4 段）。先给旅程底图：

text 一次 write 的完整旅程（时间自上而下）

```
[App 进程] [audioserver 进程]
AudioTrack.write(:2374)
→ obtainBuffer(:2254) ──cblk 共享内存──► AudioMixer/Track::getNextBuffer(:931)
│ （找一块可写环形区） （无 binder） │
├─ memcpy 用户数据→环形区（拷贝1） ├─ prepareTracks_l(:3885)
└─ releaseBuffer(:2329) ├─ threadLoop_mix(:5034)（拷贝2：混音写 sinkBuffer）
（前移写指针+事件通知） ├─ threadLoop_write(:4908→:3297)
AudioTrack.start(:888) ─binder(1次)─► Track::start(Tracks.cpp:1046) → threadLoop_write
└─ mNormalSink/mOutput->write → HAL → DAC
// 讲解注释：数据零 binder、控制一 binder——start 之后 App 与 AF 唯一的
// 联系是 cblk 里的读写指针与标志位，这是延迟与功耗的最大来源
```

8.2 App 侧：write 与它的环形缓冲

先补一个"为什么能零 binder"的伏笔：第六章 createTrack 返回的不只是句柄，还有一块共享内存——客户端 `createTrack_l` 里 `audio_track_cblk_t* cblk = static_cast<audio_track_cblk_t*>(iMemPointer)`（AudioTrack.cpp:2029）。cblk 是控制块（读写指针、标志位），紧随其后（`buffers = cblk + 1`，AudioTrack.cpp:2067）就是环形数据区；若 App 用静态 buffer 模式（`mSharedBuffer != 0`），环形区则直接映射 App 自己的 ashmem。读写指针用内存序原子操作推进，两侧进程各看各的副本——这就是"控制面 binder、数据面共享内存"的底座。

cpp frameworks/av/media/libaudioclient/AudioTrack.cpp:2374-2432（骨架+讲解注释）

```
ssize_t AudioTrack::write(const void* buffer, size_t userSize, bool blocking)
{
if (mTransfer != TRANSFER_SYNC && mTransfer != TRANSFER_SYNC_NOTIF_CALLBACK) {
return INVALID_OPERATION; // 讲解注释①：write 只服务 PUSH 模式——
// callback 模式用 onMoreData，别走这里
}
if (isDirect()) {
AutoMutex lock(mLock);
int32_t flags = android_atomic_and(
~(CBLK_UNDERRUN | CBLK_LOOP_CYCLE | ...), &mCblk->mFlags);
if (flags & CBLK_INVALID) {
return DEAD_OBJECT; // 讲解注释②：Direct 流每次写前先清 underrun
} // 标志——标志位已失效则要求重建轨道
}
size_t written = 0;
Buffer audioBuffer;
while (userSize >= mFrameSize) {
audioBuffer.frameCount = userSize / mFrameSize;
status_t err = obtainBuffer(&audioBuffer,
blocking ? &ClientProxy::kForever : &ClientProxy::kNonBlocking);
```

```
// 讲解注释③：找"可写的那段环形区"。阻塞模式
// 等到永远；非阻塞立即返回 WOULD_BLOCK——
// 游戏主循环通常自己选后者
if (err < 0) {
if (written > 0) { break; }
if (err == TIMED_OUT || err == -EINTR) { err = WOULD_BLOCK; }
return ssize_t(err); // 讲解注释④：三种返回语义要分清——
// WOULD_BLOCK=满（健康）、DEAD_OBJECT=
// 服务或路由失效（要 restoreTrack_l）
}
size_t toWrite = audioBuffer.size();
memcpy(audioBuffer.raw, buffer, toWrite); // 讲解注释⑤：唯一一次数据拷贝发生在
// 这里：用户数组→cblk 环形区
buffer = ((const char *) buffer) + toWrite;
userSize -= toWrite;
written += toWrite;
releaseBuffer(&audioBuffer); // 讲解注释⑥：前移写指针——至此数据对 AF
// 可见，但没有任何通知发送（靠指针）
}
if (written > 0) {
mFramesWritten += written / mFrameSize;
if (mTransfer == TRANSFER_SYNC_NOTIF_CALLBACK) {
...
t->wake(); // 讲解注释⑦：带回调的 SYNC 模式顺带唤醒
} // AudioTrackThread 问"还有更多数据吗"
}
return written;
}
```

设计洞察 · 逐段精读：为什么 write 是 while 循环

环形缓冲是"定容"的：一次 write 2MB 往往远超缓冲容量，循环必须多轮"取一块→拷一块→放一块"，直到用户数据耗尽或缓冲满返回。这也解释了 write 的调用语义：它可能阻塞到 AF 消费出空位——即 App 的写入节奏天然被混音周期背压。写快了等（缓冲满）、写慢了 underrun（下一节）——20ms 周期就是这条"节拍器"的刻度。

8.3 服务侧：从 start 到 HAL

数据在环形区里躺着，真正让它"走"的是 start。`Track::start`（Tracks.cpp:1046）先做 offload 反悔检查（挂了不可 offload 的全局音效直接 invalidate），再把状态机推进到 `ACTIVE`，最后 `playbackThread->addTrack_l(this)` 把自己挂进线程的 mActiveTracks——第七章 threadLoop 第 2 步 prepareTracks_l 从此开始盘它。

AF 侧真正写 HAL 的代码是 `PlaybackThread::threadLoop_write`（Threads.cpp:3297）：

cpp frameworks/av/services/audioflinger/Threads.cpp:3297-3362（骨架+讲解注释）

```
ssize_t AudioFlinger::PlaybackThread::threadLoop_write()
{
mInWrite = true;
const size_t offset = mCurrentWriteLength - mBytesRemaining;
if (mNormalSink != 0) {
const size_t count = mBytesRemaining / mFrameSize;
```

```
ATRACE_BEGIN("write");
uint32_t screenState = AudioFlinger::mScreenState;
if (screenState != mScreenState) {
mScreenState = screenState;
MonoPipe *pipe = (MonoPipe *)mPipeSink.get();
if (pipe != NULL) {
pipe->setAvgFrames((mScreenState & 1) ?
(pipe->maxFrames() * 7) / 8 : mNormalFrameCount * 2);
}
} // 讲解注释①：灭屏时收紧 MonoPipe 平均深度
// ——省电优先，延迟容忍（1.x 章 1.4 的
// "延迟用功耗换"反过来用了一次）
ssize_t framesWritten = mNormalSink->write((char *)mSinkBuffer + offset, count);
ATRACE_END(); // 讲解注释②：MixerThread 路径——写的是
// MonoPipe，数据交给 FastMixer 精混后
// 再进 HAL（第七章 7.3 的接力）
} else {
// Direct output and offload threads
if (mUseAsyncWrite) {
...
mCallbackThread->setWriteBlocked(mWriteAckSequence);
}
bytesWritten = mOutput->write((char *)mSinkBuffer + offset, mBytesRemaining);
// 讲解注释③：Direct/Offload 路径——不经
// MonoPipe，sinkBuffer 直接投 HAL；
// 异步写模式用序列号对账完成时机
...
}
mNumWrites++;
mInWrite = false; // 讲解注释④：写完不一定出声——HAL 内部
... // 还有自己的环形缓冲与 DSP 延迟
return bytesWritten;
}
```

两条路径的分岔解释了一个常见 dumpsys 现象：MixerThread 线程上 `threadLoop_write` 的耗时分两段（普通混音入 MonoPipe + FastMixer 出 HAL），Direct/Offload 线程只有一段。systrace 里 `write` 这个 ATRACE tag 的时长因此含义不同——分析延迟问题时先分清线程类型再量刻度。

8.4 出声之后：underrun 的三种症状

消费端比生产端快，环形区见底，就是 underrun。处理方式分线程：MixerThread 的 prepareTracks_l 把饿了的 track 静音并把 `underruns.mBitFields.mEmpty++`；DirectOutputThread::threadLoop_mix（Threads.cpp:6397）更直白——`if (audio_has_proportional_frames(mFormat)) memset(curBuf, 0, frameCount * mFrameSize)`，用零帧垫满本轮再交给 HAL。

设计洞察 · 逐段精读：一次 write 的内存足迹与症状对照

数据从用户数组到 DAC 总共 2-3 次 memcpy：①App→cblk 环形区（write 里）；②AudioMixer→sinkBuffer（混音，PCM 必有）；③HAL 内部（厂商实现，可选）。underrun 的三种症状对号入座：咔哒声（垫零边界跳变）、断续（周期性 underrun，App 供数慢）、持续静音（track 活着但全线饿死）。修复排序：先看 dumpsys 的 underrun 计数是否增长（排障手册场景二）、再查 App 线程是否被 GC/锁卡、最后才怀疑 HAL。

设计洞察 · 与第十二章前章呼应：录音是同一张地图的倒放

RecordThread（Threads.h:1704）的 RecordTrack 用同一套 cblk 机制反着读：HAL→sinkBuffer→环形区→App 的 AudioRecord.read。全链的 binder/memcpy 次数与本章完全对称——掌握一次 write，录音链等于免费。MediaCodec 的双队列（第三章）则是同一思想在图形内存上的翻版：队列长度受控、buffer 池化、零拷贝交接。

第九章 CameraService 接入仲裁与 Camera3Device 建立

设计洞察 · 学习目标

1. 能复述 openCamera 从 CameraManager 到 Camera3Device 就绪的六段链；2. 能列举 validateClientPermissionsLocked 的五道门（受信 uid/CAMERA 权限/UID 活跃/传感器隐私/多用户白名单）；3. 能解释 handleEvictionsLocked 的仲裁规则与 API1 复用特例；4. 能说出 T 上 HIDL/AIDL 双传输对应 HidlCamera3Device/AidlCamera3Device 的分支选择

9.1 链路位置：2.1 的第一段放大到函数级

第二章 2.1 用 ::fig cam-pipeline 给了 Camera2 的四段鸟瞰，其中 `openCamera`（frameworks/base/core/java/android/hardware/camera2/CameraManager.java:825/862/927 三个重载）是最先落地的入口。它做三件事：检查 CAMERA 权限与包名（framework 层初筛，CameraService 会再查一遍——双保险）、把回调包成 `ICameraDeviceCallbacks`、经 AudioSystem 同款的 binder 一跳进入 CameraService（libcameraservice 进程 cameraserver）。服务端总入口是 `CameraService::connectHelper`（CameraService.cpp:1750），本章逐门走读；随后设备实体的建立终点在 device3/Camera3Device.cpp——直到 `initializeCommonLocked` 把四条工作线程跑起来，"open 完成"才算落定。与音频侧对照：AudioFlinger::createTrack 是"拿一张工单"，CameraService::connect 是"包下一个车间"，所以相机的接入检查比音频重得多——摄像头是不可多占的资源。

9.2 connectHelper 的九重门

cpp frameworks/av/services/camera/libcameraservice/CameraService.cpp:1750-1930（骨架+讲解注释）

```
template<class CALLBACK, class CLIENT>
```

```
Status CameraService::connectHelper(const sp<CALLBACK>& cameraCb, const String8& cameraId,
int api1CameraId, const String16& clientPackageName, bool systemNativeClient, ...,
apiLevel effectiveApiLevel, ..., /*out*/sp<CLIENT>& device) {
binder::Status ret = binder::Status::ok();
...
{ // AutoConditionLock scope
// 门1：连接风暴限流——同时 connect 的客户端数量有上限
auto clientDescriptor = ...;
if (tooManyConnecting) {
ALOGE("CameraService::connect (PID %d) rejected (too many other clients connecting).");
return STATUS_ERROR_FMT(ERROR_MAX_CAMERAS_IN_USE, ...);
} // 讲解注释①：mServiceLockWrapper 条件锁——把并发 connect 排队串行化
// 门2+3：合法性+权限总闸
if(!(ret = validateConnectLocked(cameraId, clientName8, clientUid, clientPid,
originalClientPid)).isOk()) {
return ret; // 讲解注释②：:1174——HAL 模块加载/相机 ID 存在性
// 检查+9.3 的五道权限门
}
// 门4：仲裁——要不要踢掉现有客户端
if ((err = handleEvictionsLocked(cameraId, originalClientPid, effectiveApiLevel,
remoteCallback, clientName8, oomScoreOffset, systemNativeClient,
/*out*/&clientTmp, /*out*/&partial)).isOk() && clientTmp != nullptr) {
device = static_cast<CLIENT*>(clientTmp.get());
return ret; // 讲解注释③：:1379——同包名/更高优先级的客户端
// 命中现有会话时直接"认领"而不是重开
}
...
}
...
// 门5：从 ProviderManager 拿设备版本与传输方式
std::pair<int, IPCTransport> deviceVersionAndTransport =
getDeviceVersion(cameraId, ...); // 讲解注释④：HIDL 还是 AIDL 决定门7 的类
// 门6：建 Client（API1 还是 API2 的门面类）
if(!(ret = makeClient(this, cameraCb, clientPackageName, systemNativeClient,
clientFeatureId, cameraId, api1CameraId, facing, orientation,
clientPid, clientUid, getpid(), deviceVersionAndTransport, effectiveApiLevel,
overrideForPerfClass, /*out*/&tmp)).isOk()) {
return ret; // 讲解注释⑤：:940——Camera2Client 或
// CameraDeviceClient，见 9.4
}
client = static_cast<CLIENT*>(tmp.get());
// 门7：Client 初始化=真正打开 HAL 设备
String8 monitorTags = isClientWatched(client.get()) ? mMonitorTags : String8("");
err = client->initialize(mCameraProviderManager, monitorTags);
if (err != OK) {
// 讲解注释⑥：错误码逐一映射成 StatusError——open 失败时 App 拿到的
// CameraAccessException 类型就来自这张映射表
switch(err) {
case -EBUSY: return STATUS_ERROR_FMT(ERROR_CAMERA_IN_USE, ...);
case -EUSERS: return STATUS_ERROR_FMT(ERROR_MAX_CAMERAS_IN_USE, ...);
case PERMISSION_DENIED: return STATUS_ERROR_FMT(ERROR_PERMISSION_DENIED, ...);
case -EACCES: return STATUS_ERROR_FMT(ERROR_DISABLED, ...); // 政策禁用
case -ENODEV: default: return ...ERROR_INVALID_OPERATION...;
}
}
```

```
// 门8：API1 shim 参数、旋转裁剪覆盖、隐私静音
if (mOverrideRotateAndCropMode != ANDROID_SCALER_ROTATE_AND_CROP_AUTO) { ... }
bool isCameraPrivacyEnabled = mSensorPrivacyPolicy->isCameraPrivacyEnabled();
if (client->supportsCameraMute()) {
client->setCameraMute(mOverrideCameraMuteMode || isCameraPrivacyEnabled);
} // 讲解注释⑦：传感器隐私开且 HAL 不支持静音时——直接 disconnect，宁可不开
// 门9：状态广播
...
updateStatus(StatusInternal::NOT_AVAILABLE, cameraId); // 讲解注释⑧：从此这台相机
// 对其他客户端标记占用
device = client; // 讲解注释⑨：异步回调（CameraManager 的
return ret; // StateCallback.onOpened）在此触发
}
```

设计洞察 · 逐段精读：门4 的仲裁规则

handleEvictionsLocked 的裁决基于 resource_policy::ClientDescriptor 的优先级排序：oomScoreOffset（进程 adj 修正）+ 客户端状态。SystemUI/电话等高优进程能顶掉后台拍摄的低优进程（被踢方收到 onDisconnected）；但**同包名重复 open（API1，如 MediaRecorder）会复用现有 client 而不驱逐**——CameraService.cpp:1390 注释明言这是为 MediaRecorder 兼容。这也回答了 2.2 的"高优客户端可踢低优"是怎么来的：不是策略层的决策，是 ClientDescriptor 的严格弱序比较。

9.3 五道权限门：validateClientPermissionsLocked

cpp frameworks/av/services/camera/libcameraservice/CameraService.cpp:1222-1325（骨架+讲解注释）

```
Status CameraService::validateClientPermissionsLocked(const String8& cameraId,
const String8& clientName8, int& clientUid, int& clientPid,
/*out*/int& originalClientPid) const {
int callingPid = CameraThreadState::getCallingPid();
int callingUid = CameraThreadState::getCallingUid();
// 门1：受信身份——只有 system uid 可以代第三方申报 uid/pid
if (clientUid == USE_CALLING_UID) {
clientUid = callingUid; // 讲解注释①：默认口径=调用者本人
} else if (!isTrustedCallingUid(callingUid)) {
ALOGE("... rejected (don't trust clientUid %d)", ...);
return STATUS_ERROR_FMT(ERROR_PERMISSION_DENIED, ...); // 冒充即拒
}
// 门2：CAMERA 权限——系统自身调用豁免、系统专属相机除外
if (callingPid != getpid() && (deviceKind != SystemCameraKind::SYSTEM_ONLY_CAMERA)
&& !checkPermission(sCameraPermission, clientPid, clientUid)) {
ALOGE("Permission Denial: can't use the camera pid=%d, uid=%d", ...);
return STATUS_ERROR_FMT(ERROR_PERMISSION_DENIED, ...); // 讲解注释②：:1273——
} // framework 初筛后的服务端终审
// 门3：UID 活跃度——后台应用不得开相机
if (!mUidPolicy->isUidActive(callingUid, String16(clientName8))) {
int32_t procState = mUidPolicy->getProcState(callingUid);
return STATUS_ERROR_FMT(ERROR_DISABLED,
"Caller ... cannot open camera ... from background ...");
} // 讲解注释③：对"后台拍照"的硬闸——
// 想后台拍必须前台服务+合理场景
// 门4：传感器隐私（物理开关）
if (mSensorPrivacyPolicy->isSensorPrivacyEnabled()) {
```

```
return STATUS_ERROR_FMT(ERROR_DISABLED, ...); // 讲解注释④：用户物理禁用优先级
} // 最高——任何 App 都越不过
// 门5：多用户——只放行当前前台用户的客户端
if (!doesClientHaveSystemUid() && callingPid != getpid()
&& (mAllowedUsers.find(clientUserId) == mAllowedUsers.end())) {
return STATUS_ERROR_FMT(ERROR_PERMISSION_DENIED, ...); // 讲解注释⑤：工作资料/
} // 分身用户在此被隔离
return Status::ok();
}
```

设计洞察 · 设计洞察：匿名性的边界在"申报"而不在"使用"

音频侧第六章的守则在这里复刻：USE_CALLING_UID/USE_CALLING_PID 只允许受信 uid（system/root）使用——普通 App 想匿名访问相机根本无从申报。但注意"不匿名"指的是使用可追溯（包名+uid 记录在 Client 里，AppOps/绿点数据源），而非"使用时广播全系统"：相机图像本身没有任何身份信息流给第三方。对比第三章 DRM 的匿名访问原则：密钥服务知道你是谁，解码硬件不知道你要什么。

9.4 Camera3Device 的建立与线程布阵

门5 makeClient 的分支很简单：`effectiveApiLevel == API_1` 走 `Camera2Client`（Camera1 黑箱 shim），否则 `CameraDeviceClient`（api2/CameraDeviceClient.cpp，Camera2/X 的真身）——两条路最终都汇到 Camera2ClientBase。真正的 HAL 打开发生在 `Camera2ClientBase::initializeImpl`（common/Camera2ClientBase.cpp:97）：

cpp frameworks/av/services/camera/libcameraservice/common/Camera2ClientBase.cpp:97-140（骨架+讲解注释）

```
template <typename TClientBase>
status_t Camera2ClientBase<TClientBase>::initializeImpl(TProviderPtr providerPtr,
const String8& monitorTags) {
status_t res;
// 讲解注释①：先记 AppOps——CAMERA 使用记录（绿点数据源）从这里开始记账
res = TClientBase::startCameraOps();
if (res != OK) { return res; }
IPCTransport providerTransport = IPCTransport::INVALID;
res = providerPtr->getCameraIdIPCTransport(TClientBase::mCameraIdStr.string(),
&providerTransport); // 讲解注释②：问 ProviderManager 这台相机走
// HIDL 还是 AIDL——T 的厂商碎片化现状
switch (providerTransport) {
case IPCTransport::HIDL:
mDevice = new HidlCamera3Device(TClientBase::mCameraIdStr,
mOverrideForPerfClass, mLegacyClient); // 讲解注释③：同一逻辑设备类，
break; // 两套传输外壳——策略与第九章 HAL 版本
case IPCTransport::AIDL: // 问题呼应（书1 HAL 三代）
mDevice = new AidlCamera3Device(TClientBase::mCameraIdStr,
mOverrideForPerfClass, mLegacyClient);
break;
default:
return NO_INIT;
}
res = mDevice->initialize(providerPtr, monitorTags); // 讲解注释④：openSession 下沉
if (res != OK) { return res; } // 到厂商 HAL——失败错误码沿调用链
... // 逐层上抛成门7 的映射表
return OK;
```

```
}
```

Camera3Device::initialize 的主体在 initializeCommonLocked（device3/Camera3Device.cpp:114），它布下这台设备的整支队伍：CameraServiceWatchdog（服务侧卡死看门狗）、StatusTracker 线程（注册 InflightRequests 组件——2.2 的 in-flight 计数与 dumpsys 的来源）、Camera3BufferManager、以及最关键的 RequestThread 线程（C3Dev-%s-ReqQueue）。注意 T 上的类名没有变——Camera3Device 自 N 引入以来就是 HAL3 在框架侧的唯一化身，2.3 讲的 ZSL/多摄/HDR+ 能力全部沉淀在它 2000+ 行的请求与结果管理里，而不是某个"新类"里。

> **设计洞察 · 与第十章习题的钩子：线程命名即拓扑**
> RequestThread（发请求）、StatusTracker（状态广播）、PreparerThread（提前准备流 buffer）、还有 mFrameProcessor（CDU-FrameProc，结果元数据分发）——ps -T 或 showmap 都能看到这组线程名。第二章 2.1 说的"请求流水线"不在 App 也不在 HAL，而在 RequestThread::threadLoop（device3/Camera3Device.cpp:3395）与 submitRequestsHelper（:779）之间：setRepeatingRequest 进来，先 convertMetadataListToRequestListLocked 成 RequestList，再由 mRequestThread->setRepeatingRequests 挂入重复队列——预览"无限重复"的物理载体就是这条线程的自转。

9.5 请求与结果的流转：setRepeatingRequest 之后

9.2 到 9.4 讲完"open"，本章补上"第一次出图"——把第二章 2.1 的请求流水线压到函数级。App 侧 createCaptureSession 绑定输出 Surface 后，setRepeatingRequest/capture 经 ICameraDeviceUser.submitRequest 进入 CameraDeviceClient（api2/CameraDeviceClient.cpp），落地到 Camera3Device::submitRequestsHelper（device3/Camera3Device.cpp:779）：

cpp frameworks/av/services/camera/libcameraservice/device3/Camera3Device.cpp:779-818（骨架+讲解注释）

```
status_t Camera3Device::submitRequestsHelper(
const List<const PhysicalCameraSettingsList> &requests,
const std::list<const SurfaceMap> &surfaceMaps,
bool repeating, /*out*/ int64_t *lastFrameNumber) {
...
status_t res = checkStatusOkToCaptureLocked();
if (res != OK) { return res; } // 讲解注释①：状态闸——设备未配置/错误态
// 时提交直接失败，请求不入队
RequestList requestList;
res = convertMetadataListToRequestListLocked(requests, surfaceMaps,
repeating, requestTimeNs, /*out*/&requestList);
// 讲解注释②：metadata→Request：把 SurfaceMap
// 绑定到对应输出流——一条请求=一帧的
// 完整指令（参数+各流的目标 buffer）
if (repeating) {
res = mRequestThread->setRepeatingRequests(requestList, lastFrameNumber);
// 讲解注释③：预览/录像=挂入重复队列——
// 请求线程每轮自取，直到下次覆盖
} else {
res = mRequestThread->queueRequestList(requestList, lastFrameNumber);
// 讲解注释④：拍照=单次请求入同一队列——
```

```
// 与重复请求共享 3A 节奏，天然拿到
// "按下快门那一刻的最新参数"
}
if (res == OK) {
waitUntilStateThenRelock(/*active*/true, kActiveTimeout);
// 讲解注释⑤：等设备 ACTIVE——这段等待是
// open 后首帧延迟的一部分
}
return res;
}
```

队列的消费端是 9.4 结尾点名的 RequestThread::threadLoop（device3/Camera3Device.cpp:3395）：

cpp frameworks/av/services/camera/libcameraservice/device3/Camera3Device.cpp:3395-3465（骨架+讲解注释）

```
bool Camera3Device::RequestThread::threadLoop() {
// 讲解注释①：本函数禁止持有 mInterfaceLock——disconnect 与请求线程的死锁
// 图谱（b/143513518 注释原话）就是从这里撕开的口子
if (waitIfPaused()) { return true; }
waitForNextRequestBatch(); // 讲解注释②：取下一批请求（:4170）——
// 重复队列空转、单次请求到达即醒，
// 这就是流水线的"节拍器"
...
if (updateSessionParameters(mNextRequests[0].captureRequest
->mSettingsList.begin()->metadata)) {
...
mReconfigured |= parent->reconfigureCamera(mLatestSessionParams, mStatusId);
// 讲解注释③：session 参数变更触发整管
// 重配——"改流代价大"的代码出处
}
// Prepare a batch of HAL requests and output buffers.
res = prepareHalRequests(); // 讲解注释④：为每条请求抓输出 buffer、
// 填 HAL metadata、绑流——预览流与
// still 流在此挂到同一请求的不同
// 目标 buffer 上（分流真身）
...
}
```

设计洞察 · 逐段精读：分流不在队列而在请求内

"预览流与拍照流分流"容易被误读成两条队列：实际是同一个请求队列，每条请求自带"哪个流要哪些 buffer"的 surfaceMap；still capture 只是插入一条目标含高分辨率流的单次请求，3A 与时间戳仍按帧统一——这正是 2.3 说"只有流水线架构能做 HDR+/ZSL"的微观基础：同帧同拍多流、逐帧改参数。结果回流对称：HAL 出帧经 processCaptureResult 进入 Camera3OutputUtils::sendCaptureResult（Camera3OutputUtils.cpp:217），按 requestExtras 对账后回调 App 的 onCaptureCompleted；请求下发时 in-flight 加一、结果回收减一——附录 D 实验四观察到的 spike 就是"一条带 still 流目标的请求短暂驻留"。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. **AF 与 AP 的分工边界**：用一句话说清谁决策、谁执行，并各举一个具体的函数级例子。

- **参考答案要点**：AP 决策（getOutputForAttr 裁决输出/音量表/焦点），AF 执行（MixerThread::threadLoop_mix 混音写 HAL）；决策结果是 ioHandle+portId，执行从此不再问策略。

2. **createTrack 五段式**：哪一段决定"去哪个输出"？该段失败后还会建轨吗？

- **参考答案要点**：第三段 AudioSystem::getOutputForAttr（AudioFlinger.cpp:1079）；失败即 goto Exit，不建轨、归还策略侧登记（:1219 releaseOutput）——不存在兜底线程。

3. **fast track 门槛**：任举五条准入门槛，并说明不满足时 App 的可见后果。

- **参考答案要点**：线性 PCM/声道匹配/采样率等于硬件/装了 FastMixer/slot 空闲（另有帧数与全局音效两条）；不满足时静默降级（flags 清 FAST 位），App 延迟回落普通档但无错误。

4. **OffloadThread 的归属与省电**：为什么继承 DirectOutputThread 而不是 MixerThread？省电机制是什么？

- **参考答案要点**：压缩流不解码不混音，天然直通；省电来自 DSP 解码+CPU 可睡；代价是不能混音与 seek 延迟，全局音效可令 AP 拒开 offload（openDirectOutput）。

5. **write 全链计数**：一次 AudioTrack.write 到出声，binder 与 memcpy 各几次？各在什么位置？

- **参考答案要点**：数据零 binder（cblk 共享内存），控制面仅 start/stop 各一次 binder；memcpy 至少 2 次（App→环形区、混音→sinkBuffer），HAL 内部可能第 3 次。

6. **write 返回值语义**：WOULD_BLOCK 与 DEAD_OBJECT 各代表什么，分别怎么处理？

- **参考答案要点**：WOULD_BLOCK=环形区满（健康背压，稍后再写或加缓冲）；DEAD_OBJECT=控制块失效（路由切换/服务重启），需 restoreTrack_l 重建。

7. **underrun 症状与垫底**：三种症状是什么？DirectOutputThread 如何处理？

- **参考答案要点**：咔哒（垫零边界跳变）、断续（周期性饿）、静音（持续饿）；Direct 路径在 threadLoop_mix（Threads.cpp:6397）对线性 PCM memset 零帧垫满当轮。

8. **connectHelper 的门**：validateConnectLocked 覆盖九重门中的哪几道？

- **参考答案要点**：覆盖门2/门3——HAL 模块已加载、相机 ID 存在可用、以及内部调用 validateClientPermissionsLocked（:1222）完成权限五道门；限流与仲裁另在门1/门4。

9. **权限五道门**：各挡住什么场景？哪一道对"后台拍照"有效？

- **参考答案要点**：受信 uid 挡冒充、CAMERA 权限挡无权调用、UID 活跃挡后台进程（mUidPolicy->isUidActive，CameraService.cpp:1288）、传感器隐私挡物理禁用、mAllowedUsers 挡多用户越权；后台拍照主要被门3 拦截。

10. **HIDL/AIDL 双设备**：同一台相机为什么可能 new 出两种 Camera3Device？由什么决定？

- **参考答案要点**：由 ProviderManager 的 getCameraIdIPCTransport 返回值决定（Camera2ClientBase.cpp:110）；厂商 HAL 迁移进度不一，HidlCamera3Device/AidlCamera3Device 是同一逻辑类的传输外壳。

第二级：分析题（检验机制理解）

11. **追踪调用链**：从 AudioTrack.set() 到 MixerThread 起转，画出含跨进程跳的完整图，标注每次 binder 与共享内存交接点。

- **参考答案要点**：set→createTrack_l（AudioTrack.cpp:1883）→binder 1 次→createTrack（:1023）→getOutputForAttr（进程内到 AP）→createTrack_l（Threads.cpp:2229）→返回 TrackHandle→App start（binder 1 次）→Track::start→addTrack_l→threadLoop 自转；之后数据面零 binder。

12. **失败推演（offload 反悔）**：getOutputForAttr 因非 offloadable 全局音效拒绝 DIRECT/OFFLOAD 请求时，App 与用户体验如何？哪些 dumpsys 字段可证实？

- **参考答案要点**：回退共享 MixerThread（普通路径），音质/功耗变化但播放不中断；dumpsys media.audio_policy 显示该流落在 mixed output，AF 侧 track flags 无 OFFLOAD 位。

13. **失败推演（FastMixer 降级）**：全局音效使 fast track 全军覆没后，延迟与功耗走向如何？prepareTracks_l/FastMixer 侧各有什么证据？

- **参考答案要点**：延迟回到 normal 档（100ms 级），SCHED_FIFO 小周期空转被叫停（或退化为无 fast track 的 slot 空置）；dumpsys 的 fast track 区域无活动 track，underruns 不再累计于 fast 索引。

14. **身份校验风格对比**：AF 对冒充 uid 是"改写"，CameraService 对冒充 uid 是"拒绝"，差异的合理性是什么？

- **参考答案要点**：音频记录归属可事后纠正（attribution 语义），改写不影响安全（谁写的混音无所谓）；相机是敏感设备，申报错误身份=访问决策错误，必须 fail-fast。

15. **跨册串联（书3）**：两条接入链各经过几次 binder？各自最贵的一跳是哪次？

- **参考答案要点**：createTrack 链 1 次主 binder（App→AF）+AF→AP 走 AudioSystem 的 binder（同进程缓存可复用）；connect 链 App→CameraService 1 次、CameraService→ProviderManager/HAL 1 次；最贵一跳都是"首次 binder 触发的上下文切换与权限检查"，后续周期均无 binder。

16. **standby 推演**：从 prepareTracks_l 返回 MIXER_TRACKS_DISABLED 到线程 standby，逐步推演状态变化与可观测证据。

- **参考答案要点**：sleepTime 逐级拉长（threadLoop_sleepTime）→mStandbyTimeNs 超时→threadLoop_standby（:5006，HAL 进 standby、释放 wake lock）→mStandby=true；证据：logcat 的 AudioFlinger standby 行、dumpsys 的 Standby 标志、功耗统计的 audio 桶清零。

17. **双队列 vs 环形缓冲**：MediaCodec 双队列与 cblk 环形缓冲同为生产者-消费者，比较 buffer 归属、交接方式、背压机制。

- **参考答案要点**：归属：MediaCodec buffer 池属组件（index 租借）vs cblk 属 track；交接：显式 queue/dequeue + AsyncNotify vs 原子读写指针；背压：队列满 dequeue 返回 -1 vs obtainBuffer 阻塞/WOULD_BLOCK。

第三级：设计题（检验迁移与权衡）

18. **架构演进**：若把 AF 的"每输出一线程"模型改为单线程图调度（类似桌面系统的 AudioGraph），列出必须保持的对外承诺与可放弃的自由度。

- **参考答案要点**：承诺：三档延迟语义不变、FAST 资格制行为可预期、cblk ABI 兼容（App 生态）；可放弃：per-thread 锁域（改全局图锁）、单输出单线程的实时隔离、dumpsys 表现形态。核心权衡：图调度收益在复杂路由，代价是调度抖动全局化。

19. **排障方案**：为"插耳机后声音仍走扬声器"设计分层定位方案。

- **参考答案要点**：策略层先 dumpsys media.audio_policy 查 connected devices 是否含耳机与 selected device；AF 层 dumpsys media.audio_flinger 查该线程 Output 设备与 patch；协议栈层（蓝牙）查 A2DP profile 连接与 delay report；修复按层归位：重插拔触发重路由/恢复默认输出策略/重连 profile。

20. **权限设计**：为"App 后台合法拍摄"（如无障碍扫描）设计一条既过 validateClientPermissionsLocked 又可被用户感知/撤销的通道，指出最小改动集。

- **参考答案要点**：不可绕门3（UID 活跃），必须要求前台服务+CAMERA 权限+AppOps CAMERA 记录（绿点/通知栏持续提示天然满足感知）；可撤销复用 sensor privacy 与 AppOps 强制拒绝；最小改动集为零——现有通道已支持，验证路径为 CameraService.cpp:1288 的 isUidActive 对前台服务 uid 的放行逻辑。

术语表

术语	解释
AAudio	8.0 引入的新音频 API：stream builder+状态机，MMAP no-IRQ 低延迟路径的官方门面，底层仍由 AF/AP 管理

术语	解释
AppOps	应用操作审计框架：相机/录音的"使用记录"数据源，绿点与"最近使用"列表由此驱动
Binder 句柄（TrackHandle）	AF 把线程内 Track 包装成可跨进程 IAudioTrack 的门面类（Tracks.cpp:333），壳亡则轨灭
cblk 控制块	audio_track_cblk_t：track 的共享内存头，含原子读写指针与标志位；紧随其后是环形数据区（AudioTrack.cpp:2029）
Codec2	组件链式新一代编解码框架，参数运行时可调，T 上与 OpenMAX 并存迁移中
产品策略（ProductStrategy）	AP 引擎对一组 usage 的路由/音量决策单元：getOutputForAttr 时按 attributes 定死，Client 描述符记录在案
Direct 输出	不经混音直通 HAL 的独占输出：HDMI 多声道/低延迟单流，由 openDirectOutput 按 profile 开设
DuplicatingThread	MixerThread 子类：混音一次、复制给多个下游 OutputTrack，实现双设备齐播（Threads.h:1627）
FastMixer	MixerThread 内的 SCHED_FIFO 小周期精混线程，经无锁 MonoPipe/StateQueue 与普通混音线程解耦（FastMixer.cpp）
Fast Track	fast mixer 的有限名额轨：资格制准入（线性 PCM/参数一致/slot 空闲等七条），终裁在 createTrack_l（Threads.cpp:2282）
HAL3	相机 HAL 第三代接口：请求-结果模型+3A 可编程，framework 侧化身 Camera3Device
混音周期	PlaybackThread 一轮 prepare→mix→write 的时间刻度：frameCount/sampleRate，深管 20ms 级
环形缓冲	cblk 数据区的 FIFO 结构：写满则 obtainBuffer 阻塞，读空则 underrun 垫零
in-flight	相机已下发未回结果的请求集合：StatusTracker 的 InflightRequests 组件监控，dumpsys media.camera 可见
焦点（AudioFocus）	音频注意力的约定仲裁：AP 裁决并广播 LOSS/GAIN，混音层不物理停流
快门回溯	ZSL 的行为表述：环形缓冲持续捕帧，按快门时刻回溯取帧，依赖逐帧可控的流水线
流组合	HAL3 声明的 stream configuration 支持集：预览+录像+分析等 Surface 组合是否合法以 HAL 能力表为准
MixerThread	多轨混音线程：多 Track→一路 PCM→HAL，可选装配 FastMixer（Threads.h:1393）

术语	解释
MMAP no-IRQ	共享内存直投+硬件时钟驱动的最低延迟路径（8+）：AAudio 首选，MmapThread 承载
匿名客户端检查	USE_CALLING_UID/USE_CALLING_PID 仅受信 uid 可用的信任边界：AF 改写、CameraService 拒绝
Offload	压缩流（MP3/AAC）不解码直送 DSP 的输出档：省电但不可混音，被全局音效阻断时回退普通路径
portId	AP 发放的客户端级句柄：一次接入一个，dumpsys 客户端表的主键
prepareTracks_l	混音周期的裁判函数（Threads.cpp:3885 调用）：汇总 track 状态，产出 MIXER_TRACKS_READY/DISABLED
请求-结果对	Camera2 核心抽象：每帧一个 CaptureRequest，回 TotalCaptureResult（图像+拍摄参数）
双队列	MediaCodec 的 input/output buffer 池模型：App 与编解码器经队列交接，buffer 池化复用
underrun	消费快于生产的饥饿状态：症状为咔哒/断续/静音，dumpsys 的 underruns 字段累计计数
音量表	STREAM 类型×设备的音量矩阵：setStreamVolume 改表，真实增益由 AF 在混音时施加
ZSL	Zero Shutter Lag：零快门延迟拍摄，快门回溯的机制名（2.3）

排障手册：媒体管线问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。

10.1 音频延迟实测教程

```bash
# 测量延迟（tap-to-tone 原理）
# 发一个触屏事件 → 音频输出 beep → 测差值
# 1. 写一个测试 App：
# onTouchEvent → 发送系统时间戳 → AudioTrack.write(beep)
# 2. 在扬声器处放麦克风（或用另一台设备录制）
# 3. 比较触摸事件时间戳 vs 声音到达时间戳
# 典型值（高通设备实测）
# normal track: ~100-150ms（binder 写+混音+HAL 4 周期）
# fast track: ~40-60ms（共享内存+低周期）
# MMAP no-IRQ: ~10-20ms（硬件时钟直驱）
# 蓝牙 A2DP: ~200-300ms（编码+传输+解码+重采样）
# Offload: ~50-100ms（DSP 直出，视 codec 而定）
// 讲解注释①：latency = buffer_period × 2（输入侧+输出侧）+ 硬件处理时间
// 讲解注释②：游戏/音游选 MMAP；正常通知选 normal 即可——延迟是有代价的
```

10.2 播放无声但进度在走

症状：播放器进度正常、回调正常，扬声器/耳机无任何声音；通知音正常。

定位命令：`dumpsys media.audio_flinger`（看目标线程是否 ACTIVE、track 的 Usage/Volume/Fmt 是否合理）；`dumpsys media.audio_policy`（看该 usage 路由到哪个设备、客户端表里 portId 是否在场）；`logcat -s AudioFlinger AudioPolicyManager`（建轨失败/路由切换日志）。

根因：三类常见——①路由到了不存在的设备（耳机断连后策略表未回切，输出指向已移除的 device）；②软件音量/静音挡位（STREAM 音量为 0 或被 DND 压制，AF 在混音时乘 0 增益）；③track 被焦点方 duck 到 0 增益但从未恢复（1.2 的"约定"未被守约方恢复）。

修复：按根因归位——策略表异常时切换输出源触发重路由（或重启 audioserver 验证表是否重建）；音量表问题确认音量源（第六章 6.3 的 toVolumeSource 归属）；duck 未恢复时检查 duck 的发起方是否遗留（焦点栈）。

10.3 声音断续/咔哒（underrun）

症状：播放中周期性断续或咔哒声，负载高时加剧；换播放器/App 无改善。

定位命令：`dumpsys media.audio_flinger`（看目标 track 的 Underruns 计数是否持续增长）；`logcat -s AudioFlinger`（underrun 与 "AUDIO_OUTPUT_FLAG_FAST denied" 行）；`simpleperf` 或 systrace（看 App 写入线程与 AF 混音线程的调度占用）。

根因：①App 供数不足——写入线程被 GC/锁/网络 IO 卡住，环形区见底（第八章 8.4）；②CPU 级饥饿——混音线程（SCHED_FIFO）被高负载挤占或 effect 链超预算；③fast track 名额耗尽后 low-latency 期望落空，实际跑在 normal 档但 App 按 fast 节奏写（背压被误解）。

修复：①加缓冲/独立写线程（写入与业务解耦）；②排查拖垮实时调度的 effect 或第三方 hook，必要时关闭特效；③以 dumpsys 里 track 实际 flags 为准重设延迟预期（第七章 7.3 的"资格制"）。

10.4 插耳机后声音仍走扬声器

症状：有线耳机/蓝牙连接成功，部分声音（或全部）仍从扬声器出；或反过来全部声音被"吸入"耳机。

定位命令：`dumpsys media.audio_policy`（connected devices 与各流的 selected device，注意通话/媒体可能不同设备）；`dumpsys media.audio_flinger`（各线程 Output 字段对应的 device type）；`adb shell cmd media_session` 或蓝牙栈日志（A2DP profile 连接状态与 delay report）。

根因：①策略未重路由——设备插拔事件未到达 AP（HAL 侧 device connection 回调缺失）；②forced use 生效——通话强制走耳机/免提的 FORCE 值未清除，影响同 usage 的媒体；③蓝牙 A2DP 已连接但音频未起播（profile 未 ready 时 AP 已把设备标记为可用）。

修复：①重新插拔或 `adb shell cmd audio` 触发设备重连验证回调链；②检查 forceUse 设置源（通话状态机）；③蓝牙场景先断连重连 A2DP profile，再查书10 对应的协议栈状态。

10.5 相机 open 失败

症状：App 收到 CameraAccessException；错误码与 openCamera 失败路径对应（第九章 9.2 门7 的映射表）。

定位命令：`dumpsys media.camera`（Device status 与 active clients：谁占用、优先级多少）；`appops get <pkg> CAMERA`（该包使用记录与权限状态）；`logcat -s CameraService`（validateClientPermissionsLocked 的具体拒绝行：Permission Denial/Access Denial 分支）。

根因：①ERROR_CAMERA_IN_USE——另一客户端占用（含 SystemUI 人脸/扫码类常驻客户端），高优仲裁踢人失败；②ERROR_PERMISSION_DENIED——CAMERA 权限缺失或非受信代申报（CameraService.cpp:1273）；③ERROR_DISABLED——UID 不活跃（后台）或传感器隐私开启（CameraService.cpp:1288 起）；④ERROR_MAX_CAMERAS_IN_USE——连接风暴限流或多摄占用上限。

修复：①占用问题先找到占用方（dumpsys 的 client 表），必要时重置占用（关闭常驻扫码）或提升本进程优先级走仲裁；②权限问题到设置页授权或修正 manifest；③后台问题改为前台服务并等待 UID 活跃；④限流问题排查 App 是否并发 connect 泄漏 client（onDisconnected 后未释放）。

10.6 预览黑屏（但 open 成功）

症状：onOpened 回调已到、setRepeatingRequest 也成功，Surface 上无画面。

定位命令：`dumpsys media.camera`（Session 的 stream 表：宽高/格式/最大帧率是否被 HAL 接受；in-flight 是否在涨）；`logcat -s Camera3-Device Camera3-Stream`（configure 失败/dequeue buffer 超时）；`dumpsys SurfaceFlinger`（Surface 是否有 buffer 投递、图层可见性）。

根因：①流组合不被 HAL 支持——surface 尺寸/格式不在能力表（第二章 2.2 的"流组合三查"）；②Surface 消费端死亡——TextureView/SurfaceView 已 detach 但未重建会话；③3A/曝光被 App 请求参数锁死（AE off+极短曝光=全黑画面，画面在走但没有内容）。

修复：①按 HAL 能力表重选 supported size（结合 2.3 的能力查询）；②重建会话（createCaptureSession 换新 Surface）；③核对请求模板与手动 AE 参数，先用 TEMPLATE_PREVIEW 排除参数因素再逐项收敛。

10.7 录制卡顿/掉帧

症状：预览流畅，MediaRecorder 或 ImageReader 录制产出视频丢帧、时间戳跳变。

定位命令：`dumpsys media.camera`（录制会话的 in-flight 峰值与 result drop 计数）；`dumpsys media.codec`（编码器实例状态、输入/输出 buffer 积压）；`logcat -s CCodec VideoCapabilities`（编码器配置与吞吐告警）。

根因：①still+preview+录像多流并发超出 ISP 带宽——HAL 降帧率或丢流（高分辨率传感器单摄模式常见，参考 2.3 的多摄裁剪）；②编码器吞吐不足——软件编解码被选上（Codec2 组件链降级）或码率/分辨率组合超载；③ Surface 队列满——消费者（编码器/存储）慢，图形 buffer 耗尽反压回 HAL。

修复：①降流（裁掉分析流/降低 still 尺寸）或改用多摄高分辨率模式；②核对编码器为硬件实现与 profile 等级，必要时降分辨率档；③检查存储介质写入速度（SD 卡场景先行），并为 SurfaceFlinger 侧确认无重复合成压力。

媒体管线核心速查表

机制	一句话	源码锚点	所在册
AF 混音车间	MixerThread(:4908) 周期混音写 HAL	AudioFlinger.cpp	本册
AP 调度室	路由/音量/焦点（getOutputForAttr）	AudioPolicyManager	本册
三档延迟	normal(~100ms)/fast(~20ms)/MMAP(<10ms)	AudioTrack/AAudio	本册
焦点约定	策略裁决+广播 LOSS/GAIN（混音不拦）	AudioPolicyManager	本册
Offload 省电	压缩流直送 DSP（CPU 可睡）	AF/OffloadThread	本册
Camera2 抽象	异步请求-结果对（请求队列+in-flight）	CameraService.cpp:1750	本册
ZSL	环形缓冲持续捕帧→按快门时回溯取帧	Camera2 流水线	本册
MediaCodec 双队列	App↔inputBuffer↔codec↔outputBuffer↔App	MediaCodec	本册
Codec2	组件链式新一代编解码框架	frameworks/av/media/libstagefright	本册
MediaDrm L1/L3	硬件全链(4K) vs 软件受限	frameworks/av/drm	本册